Formal Semantics and Theorem Reference for Mergeable Replicated Data Types

Anonymous Authors

Abstract

This document gives a bottom-up reference for the public formal theory of mergeable replicated data types (MRDTs) mechanized in Lean 4. It starts with events and datatype signatures, defines the version-DAG operational semantics, then explains how finite event histories determine stored states, which merge property preserves that explanation, and how operation-generation constraints connect it to an independent client specification. It also covers merge-base construction for version DAGs without a single greatest common ancestor and two independent garbage-collection refinements. Definitions are ordered by semantic dependence rather than Lean file order. Every principal definition and theorem has its exact Lean declaration and a source link. Proof-local lemmas are omitted from the main line; the historical binary development is isolated in an appendix.

Contents

1 Purpose and evidence boundary

This document is written for readers who want to understand the formal theory before navigating its Lean implementation. It reconstructs the interface and theorem dependency graph and then instantiates them for every released RDT. It answers four questions:

1. What are the exact states and transitions of the MRDT machine?
2. What invariant makes every stored version replayable, and what obligation preserves it at merge?
3. How is internal replay connected to an independent client specification?
4. Which history- and state-collection steps refine the same uncollected semantics?

The top-level result is client-facing: every released datatype packages an issuance discipline, a merge-adequacy proof, an independent sequential specification, and a refinement proof. Consequently, every version produced by an issuance-certified ordinary or virtual-merge-base execution admits a legal sequential history with the same events and the same query results. Sections 2–8 define the layers needed by this statement. Section 9 shows how each released instance supplies them. Sections 10–11 add optional history- and datatype-state collection refinements.

Reader's map.

Layer	Contract and consequence	Location
Operational machine	Events and the datatype signature determine fork, apply, query, and ternary merge in a version DAG.	Sections 2–3
Replay model	A replay context orders a finite event set and gives it a unique canonical state.	Sections 4–5
Merge preservation	An all-context or restricted merge theorem preserves that canonical state at a merge step.	Section 6
Client correctness	Issuance, public interaction order, an independent sequential machine, and a representation relation turn internal replay into observable sequential correctness.	Section 7
Virtual bases	A recursive fold over maximal common ancestors constructs the canonical intersection state when no stored GCA exists.	Section 8
Released instances	Each package records its representation, issuance policy, merge theorem, sequential specification, relation, and proof route.	Section 9
Collection	Commit-history and datatype-state collection refine the same uncollected semantics and compose by trace erasure; the datatype profiles state what each released representation can presently reclaim.	Sections 10–11

The mathematical displays are transcriptions or notation-level expansions of the cited declarations. The trusted computing base for machine-checked statements is Lean's kernel plus Mathlib. The transcription, notation, prose, and links remain review obligations. No claim is made that a separately implemented runtime is extracted from Lean.

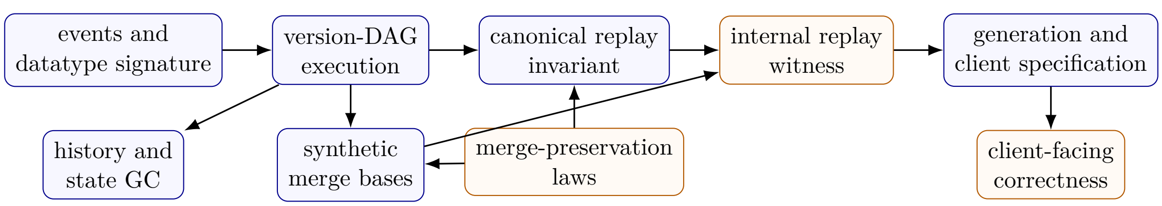

Figure 1: Semantic dependency order. Arrows point from prerequisite to consumer. The two GC layers are optional refinements, not premises of client-facing correctness.

2 Primitive types and datatype signatures

2.1 Identifiers and events

Replicas, timestamps, and versions are natural numbers:

$$\mathsf{Replica} = \mathbb{N}, \qquad \mathsf{Timestamp} = \mathbb{N}, \qquad \mathsf{Version} = \mathbb{N}.$$

For an application-operation type O, an event is

$$\mathsf{Event}(O) = \mathsf{Timestamp} \times \mathsf{Replica} \times O.$$

If $e = (t, r, o)$, write $\mathsf{time}(e) = t$ and $\mathsf{rep}(e) = r$. Timestamps are Lamport timestamps. Global uniqueness and causal monotonicity are properties of valid configurations and the apply transition, not of the bare product type.

Lean: `Foundation.Replica, Foundation.Timestamp, Foundation.Op`.

Definition 2.1 (MRDT signature)**.** An MRDT signature is

$$D = (\Sigma, \sigma_0, O, Q, R, \mathsf{do}, \mathsf{merge}, \mathsf{query})$$

with decidable equality on Σ and O, and

$$\begin{aligned} \mathsf{do} &: \Sigma \to \mathsf{Event}(O) \to \Sigma, \\ \mathsf{merge} &: \Sigma \to \Sigma \to \Sigma \to \Sigma, \\ \mathsf{query} &: \Sigma \to Q \to R. \end{aligned}$$

The intended reading of $\mathsf{merge}(l, a, b)$ is a three-way merge with ancestor state l and branch states a, b. The ancestor is semantically necessary for datatypes whose state combines by addition rather than idempotent union: the flat counter, for example, uses $\mathsf{merge}(l, a, b) = a + b - l$ so that effects already present in both branches are counted once. Ternary merge covers both these group-like states and ordinary grow-only unions with one interface. The paper's `do` is the Lean field `MRDTSig.update`; Lean reserves `do` for its term syntax.

Lean: `MRDTSig`.

This signature is the executable datatype interface. Verification attaches replay, merge, generation, and client certificates to this same D.

Running example: tombstone RGA. The production RGA stores grow-only insertion and deletion facts:

$$\begin{aligned} \Sigma_{\text{RGA}} &= (\mathbb{N}^3 \to \mathsf{Bool}) \times (\mathbb{N} \to \mathsf{Bool}), \\ O_{\text{RGA}} &= \mathsf{addAfter}(\mathit{anchor}, \mathit{id}) \mid \mathsf{remove}(\mathit{id}). \end{aligned}$$

An insertion event sets the bit indexed by its timestamp, anchor, and element identifier; a removal sets the tombstone bit for its identifier. Merge is pointwise Boolean union of the GCA and two branch states. The query sorts the finite insertion support, inserts each identifier after its anchor, and filters tombstoned identifiers. Thus the implementation state is unordered grow-only evidence, while the client observes an ordinary list.

Lean: `RGA.RGAM, RGA.rgaUpdate, RGA.read`.

2.2 Finite replay

Finite replay uses only the merge-free projection

$$D_U = (\Sigma, \sigma_0, O, \mathsf{do}).$$

Lean calls this derived structure `UpdateSig`; it retains precisely the fields used when an event history is folded.

For $\pi = [e_1, \ldots, e_n]$,

$$\mathsf{applySeq}(D, s, \pi) = \mathsf{do}(\cdots \mathsf{do}(\mathsf{do}(s, e_1), e_2) \cdots, e_n).$$

A list enumerates a set exactly when

$$\mathsf{listPermOf}(\pi, E) \iff \mathsf{Nodup}(\pi) \wedge \forall e.\ (e \in \pi \leftrightarrow e \in E).$$

It respects a relation R when no later element is required to precede an earlier one:

$$\mathsf{respects}(\pi, R) \iff \mathsf{Pairwise}_\pi(\lambda a\ b.\ \neg R(b, a)).$$

Lean: `Foundation.applySeq, Foundation.listPermOf, Foundation.respects`.

Two events commute when they commute from every state:

$$e_1 \leftrightarrow e_2 \iff \forall s : \Sigma.\ \mathsf{do}(\mathsf{do}(s, e_1), e_2) = \mathsf{do}(\mathsf{do}(s, e_2), e_1).$$

A proof-local replay policy has type

$$\mathsf{order} : \mathsf{Event}(O) \to \mathsf{Event}(O) \to \{\mathsf{FstThenSnd}, \mathsf{SndThenFst}, \mathsf{Either}\}.$$

The default policy returns Either. This policy is used only by the internal replay proof in Section 4. Client-visible interaction ordering is defined independently in Section 7. Every datatype in the production registry uses this default policy at its raw MRDT layer. Hence released datatypes have no policy-selected raw replay edges; Section 4 shows that the remaining directional law forces distinct cross-replica raw events to commute. The non-default policy in the OR-Set development belongs to its abstract sequential set machine, where it selects add-wins order. Finite replay, commutation, and the proof policy are factored through the merge-free projection of D.

Lean: `Foundation.UpdateSig.commutes, Foundation.RcRes, Foundation.ReplayPolicy`.

Lean: `Production.registry`.

3 Version-DAG operational semantics

3.1 Graph order

For $P : \mathsf{Version} \to \mathsf{List}(\mathsf{Version})$,

$$\mathsf{Reaches}(P, a, b) \iff (\lambda x\ y.\ x \in P(y))^*(a, b).$$

The ordinary merge rule uses a greatest-common-ancestor predicate:

$$\begin{aligned}\mathsf{IsGCA}(P, v_1, v_2, v_T) \iff & v_T \mathrel{\mathsf{Reaches}} v_1 \wedge v_T \mathrel{\mathsf{Reaches}} v_2 \\ & \wedge \forall w.\ w \mathrel{\mathsf{Reaches}} v_1 \to w \mathrel{\mathsf{Reaches}} v_2 \to w \mathrel{\mathsf{Reaches}} v_T.\end{aligned}$$

Every common ancestor reaches a GCA. Ranked reachability is antisymmetric, so a GCA is unique when it exists. A criss-cross DAG may instead have several incomparable maximal common ancestors and no GCA; Section 8 constructs its virtual merge base.

Lean: `Reaches, IsGCA`.

Lean: `isGCA_unique`.

For a pair of versions a, b, define

$$\begin{aligned}\mathsf{CommonAncestors}(P, a, b) &= \{x \mid x \mathrel{\mathsf{Reaches}} a \wedge x \mathrel{\mathsf{Reaches}} b\}, \\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) &\iff m \in \mathsf{CommonAncestors}(P, a, b) \\ &\qquad \wedge \forall x \in \mathsf{CommonAncestors}(P, a, b).\ m \mathrel{\mathsf{Reaches}} x \to x = m.\end{aligned}$$

Thus m is maximal when no distinct common ancestor descends from it. The self-case is present because Reaches is reflexive. Several incomparable versions may satisfy this predicate.

Lean: `CommonAncestors, IsMaximalCommonAncestor`.

3.2 Configurations

For an MRDT D, write the version store as a partial map and let $A_C = \mathsf{dom}(S)$ be its allocated-version set. A configuration contains

$$\begin{array}{rcll} S & : & \mathsf{Version} \rightharpoonup (\Sigma \times \mathcal{P}(\mathsf{Event}(O))) & \text{allocated version contents,} \\ H & : & \mathsf{Replica} \rightharpoonup A_C & \text{active replica heads,} \\ P & : & \mathsf{Version} \to \mathsf{List}(\mathsf{Version}) & \text{parent lists,} \\ \mathsf{vis} & : & \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop} & \text{visibility.} \end{array}$$

Thus $C = \langle S, H, P, \mathsf{vis} \rangle$. Write

$$\begin{aligned} \mathsf{state}_C : A_C \to \Sigma, &\qquad \mathsf{state}_C(v) = \mathsf{fst}(S(v)), \\ \mathsf{events}_C : A_C \to \mathcal{P}(\mathsf{Event}(O)), &\qquad \mathsf{events}_C(v) = \mathsf{snd}(S(v)), \\ \mathsf{headState}_C = \mathsf{state}_C \circ H, &\qquad \mathsf{headEvents}_C = \mathsf{events}_C \circ H. \end{aligned}$$

The configuration's active event universe is

$$\begin{aligned} C.\mathsf{events} &\equiv \{e \mid \exists r, E.\ \mathsf{headEvents}_C(r) = E \wedge e \in E\} \\ &= \bigcup_{r \in \mathsf{dom}(H)} \mathsf{headEvents}_C(r). \end{aligned}$$

It contains the events observed at active replica heads, not the union of all historical version event sets. These functions and the active event universe are derived observations, not configuration components.

Lean: `Configuration.events`.

For any partial map f, write $f(x) = \bot \iff x \notin \mathsf{dom}(f)$.

A well-formed semantic configuration satisfies:

1. Every visibility endpoint occurs in some $\mathsf{headEvents}_C(r)$, and each head event set is closed under visibility.
2. Distinct active events have distinct timestamps; e_1 vis e_2 implies $\mathsf{time}(e_1) < \mathsf{time}(e_2)$; and distinct events from one replica are visibility-comparable.
3. Parents have smaller version numbers and $S(0) = (\sigma_0, \varnothing)$.
4. If $S(v_i) = (s_i, E_i)$ for $i \in \{1, 2, T\}$ and v_T is a GCA of v_1, v_2, then $E_T = E_1 \cap E_2$.

Thus a configuration is already an operationally well-formed state. Client safety and canonical replay are separate predicates.

Lean: `Configuration`.

Lean totalizes the two partial maps: `ver` and `head` return `Option`, while `parents` is the total function P. The `head_alloc` field implements the paper type $H : \mathsf{Replica} \rightharpoonup A_C$. The functions `Configuration.headState` and `Configuration.headEvents` implement the two partial compositions above. The store invariant in Section 3.4 connects P to A_C: every version named in a parent list must lie in A_C. Until that invariant is imposed, keeping P total avoids adding a second allocation domain.

Write $\mathsf{initConfig}(D)$ for the initial configuration. It has replica 0 at version 0, both with state σ_0 and empty event set. All other replicas and versions are absent, parent lists are empty, and visibility is empty.

Lean: `initConfig`.

3.3 Labels, freshness, and transition rules

The label constructors have explicit types

$$\begin{aligned}
\mathsf{fork} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D),\\
\mathsf{apply} &: \mathsf{Timestamp} \to \mathsf{Replica} \to O \to \mathsf{Label}(D),\\
\mathsf{merge} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D),\\
\mathsf{query} &: \mathsf{Replica} \to Q \to R \to \mathsf{Label}(D).
\end{aligned}$$

Fork is destination-first: $\mathsf{fork}(r_d, r_s)$.

Lean: `Label`.

Write $\mathsf{freshTime}(C, t)$ when no event at an allocated version has timestamp t, and $\mathsf{freshVer}(C, v')$ when $v' \notin A_C$. The condition $v < v'$ is a rank condition for the version DAG, not a timestamp comparison. It makes the parent relation well founded. The following rules display all changed semantic fields; other entries remain unchanged, and the resulting configuration satisfies the configuration conditions above.

Fork

$$\frac{\begin{array}{c} H(r_s) = v \quad \mathsf{headState}_C(r_s) = s \quad \mathsf{headEvents}_C(r_s) = E \\ H(r_d) = \bot \quad \mathsf{freshVer}(C, v') \quad v < v' \end{array}}{C \xrightarrow{\mathsf{fork}(r_d, r_s)} C\big[S(v') \mapsto (s, E), \quad H(r_d) \mapsto v', \quad P(v') \mapsto [v] \quad \big]}$$

Fork creates a fresh child of an existing source head, preserving that source as the child's parent.

Lean: `Step.fork`.

Lean: `Step.fork_copies_source, Step.fork_head_ne_root`.

Apply

$$\frac{\begin{array}{c} H(r) = v \quad \mathsf{headState}_C(r) = s \quad \mathsf{headEvents}_C(r) = E \quad e = (t, r, o) \\ \mathsf{freshTime}(C, t) \quad \mathsf{freshVer}(C, v') \quad v < v' \end{array}}{C \xrightarrow{\mathsf{apply}(t, r, o)} C \begin{bmatrix} S(v') \mapsto (\mathsf{do}(s, e), E \cup \{e\}), & H(r) \mapsto v', \\ P(v') \mapsto [v], & \mathsf{vis} \mapsto \mathsf{vis} \cup (E \times \{e\}) \end{bmatrix}}$$

Freshness gives globally unique timestamps. Because the resulting configuration also satisfies causal timestamp monotonicity, all events visible to e have smaller timestamps. Together these are the Lamport-clock conditions. The raw rule has no issuance premise.

Lean: `Step.apply`.

Query

$$\frac{\mathsf{headState}_C(r) = s \qquad z = \mathsf{query}(s, q)}{C \xrightarrow{\mathsf{query}(r, q, z)} C}$$

Query is observational.

Lean: `Step.query`.

Merge

$$\frac{\begin{array}{c} H(r_i) = v_i \quad \mathsf{headState}_C(r_i) = s_i \quad \mathsf{headEvents}_C(r_i) = E_i \qquad (i \in \{1, 2\}) \\ \mathsf{IsGCA}(P, v_1, v_2, v_T) \quad \mathsf{state}_C(v_T) = s_T \quad \mathsf{events}_C(v_T) = E_T \\ \mathsf{freshVer}(C, v_m) \quad v_1 < v_m \quad v_2 < v_m \end{array}}{C \xrightarrow{\mathsf{merge}(r_1, r_2)} C \begin{bmatrix} S(v_m) \mapsto (\mathsf{merge}(s_T, s_1, s_2), E_1 \cup E_2), & \\ H(r_1) \mapsto v_m, & P(v_m) \mapsto [v_1, v_2] \end{bmatrix}}$$

The first replica receives the merged head. Merge creates no event.

Lean: `Step.merge`.

3.4 Store invariant and GCA events

The store invariant has three components. First, define

$$\mathsf{ParentsAllocated}(S,P) \iff \forall v,p.\ p \in P(v) \Rightarrow p \in \mathsf{dom}(S).$$

Every parent named by an edge is present in the version store. Under this condition, P may be viewed as $P : \mathsf{Version} \to \mathsf{List}(A_C)$, and its restriction to allocated children has type $P|_{A_C} : A_C \to \mathsf{List}(A_C)$. Define event monotonicity along a parent edge by

$$\begin{aligned}\mathsf{EventsMonotone}(S,P) \iff \forall v,p,s_p,E_p,s_v,E_v.\ & p \in P(v) \wedge S(p) = (s_p,E_p) \wedge S(v) = (s_v,E_v)\\ & \Rightarrow E_p \subseteq E_v.\end{aligned}$$

Thus a child contains every event stored at its parent. Finally, define

$$\begin{aligned}\mathsf{EventOrigin}(S,P) \iff \forall v,s,E,e.\ & S(v) = (s,E) \wedge e \in E \Rightarrow\\ & \exists v_0,s_0,E_0.\ S(v_0) = (s_0,E_0) \wedge e \in E_0\\ & \quad \wedge \forall w,s_w,E_w.\ S(w) = (s_w,E_w) \wedge e \in E_w\\ & \qquad\qquad \Rightarrow \mathsf{Reaches}(P,v_0,w).\end{aligned}$$

Every stored event therefore has an allocated origin version containing it; that version is an ancestor of every allocated version containing the event. No uniqueness of the origin is asserted. The store invariant is the conjunction

$$\begin{aligned}\mathsf{StoreInv}(S,P) \iff & \mathsf{ParentsAllocated}(S,P)\\ & \wedge \mathsf{EventsMonotone}(S,P)\\ & \wedge \mathsf{EventOrigin}(S,P).\end{aligned}$$

These three predicates correspond respectively to Lean's `parents_alloc`, `events_mono`, and `origin` fields. The proof of the GCA event equation factors through their conjunction. Its principal consequence is

$$\begin{gathered}\mathsf{StoreInv}(S,P) \wedge \mathsf{IsGCA}(P,v_1,v_2,v_T)\\ \wedge S(v_i) = (s_i,E_i)\ (i \in \{1,2,T\}) \Longrightarrow E_T = E_1 \cap E_2.\end{gathered}$$

Lean: `StoreInv, gca_events_of_storeInv`.

4 Replay contexts and set-relative order

4.1 Replay context

For a stored event set E, the replay problem is to enumerate exactly the events in E in an admissible order and fold their updates from σ_0. Admissibility depends on which events have been observed and on their visibility relation. It does not depend on version identifiers, parent edges, materialized states, or merge bases. The replay context isolates this smaller input to the ordering proofs.

Let L : $\mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O))$ map each active replica to the event set at its current head. Here a vis b means that a is visible to b, so a causally precedes b. Define the observed event universe by

$$\mathsf{Observed}_L(e) \iff \exists r, E.\ L(r) = E \wedge e \in E.$$

The two required coherence conditions are

$$\begin{aligned}\mathsf{TimestampUnique}(L) &\iff \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b \Rightarrow \mathsf{time}(a) \neq \mathsf{time}(b),\\ \mathsf{SameReplicaTotal}(L, \mathsf{vis}) &\iff \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b\\ &\qquad\qquad\qquad \wedge \mathsf{rep}(a) = \mathsf{rep}(b) \Rightarrow a \ \mathsf{vis}\ b \vee b\ \mathsf{vis}\ a.\end{aligned}$$

Thus timestamps identify observed events globally, while visibility compares every pair of distinct events issued by one replica. A replay context is formally

$$\mathsf{ReplayContext}(D) = \left\langle \begin{array}{l} L : \mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O)),\\ \mathsf{vis}\colon \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop},\\ u_{\mathrm{ts}} : \mathsf{TimestampUnique}(L),\\ u_{\mathrm{rep}} : \mathsf{SameReplicaTotal}(L, \mathsf{vis}) \end{array} \right\rangle.$$

For a replay context $R = \langle L, \mathsf{vis}, u_{\mathrm{ts}}, u_{\mathrm{rep}} \rangle$, the event universe is the union of the active replicas' head-event sets:

$$R.\mathsf{events} = \{e \mid \mathsf{Observed}_L(e)\} = \bigcup_{r \in \mathsf{dom}(L)} L(r).$$

Every MRDT configuration C projects to a replay context by taking $L = \mathsf{headEvents}_C$ and retaining $C.\mathsf{vis}$. The configuration fields `timestamps_distinct` and `vis_total_same_replica` supply the two proofs. Lean totalizes L as an Option-valued function; `ReplayContext.L` receives the derived head-event map, and `Configuration.replayContext` constructs the projection. Write $R_C = \mathsf{replayContext}(C)$ for this projection whenever C is an MRDT configuration. The projection preserves the active event universe:

$$R_C.\mathsf{events} = C.\mathsf{events}.$$

Lean: `Foundation.ReplayContext`.

Lean: `Configuration.replayContext`.

4.2 Set-relative replay order

For replay context R, version event set E, and events e_1, e_2, define

$$\begin{aligned} e_1\ \mathsf{loOn}_{R,E}\ e_2 \iff & (e_1\ \mathsf{vis}\ e_2 \wedge \neg(e_1 \leftrightarrow e_2))\\ & \vee (\neg(e_1\ \mathsf{vis}\ e_2) \wedge \neg(e_2\ \mathsf{vis}\ e_1) \wedge \mathsf{order}(e_1, e_2) = \mathsf{FstThenSnd}\\ & \quad \wedge \neg\exists e_3 \in E.\ e_2\ \mathsf{vis}\ e_3 \wedge \neg(e_2 \leftrightarrow e_3)).\end{aligned}$$

An event $e_3 \in E$ satisfying the final existential condition is an *absorber* for e_2. Only an absorber inside E can remove an arbitration edge needed to replay E. Growing the set may add absorbers and therefore remove such edges:

$$E \subseteq E' \wedge \mathsf{loOn}_{R,E'}(a, b) \Longrightarrow \mathsf{loOn}_{R,E}(a, b).$$

The historical global order is $\mathsf{lo}_R = \mathsf{loOn}_{R,R.\mathsf{events}}$; every lo_R edge is an edge of $\mathsf{loOn}_{R,E}$. The set parameter is therefore essential in the general theory: whether an arbitration edge is needed depends on whether its absorber belongs to the particular version being replayed, not merely on the global event universe.

Lean: `Foundation.loOn, Foundation.loOn_mono, Foundation.loOn_of_lo`.

For the default Either policy used by production MRDT signatures, the arbitration disjunct is false and the definition simplifies to

$$a \ \mathsf{loOn}_{R,E}\ b \iff a \ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b).$$

The general absorber machinery remains part of the reusable replay theorem, but it is dormant for the current production registry.

RGA instance. RGA insertion and tombstone updates commute at the raw storage layer: two insertions set Boolean insertion facts, two removals set tombstone facts, and an insertion and removal touch different components. With the default policy, $\mathsf{loOn}_{R,E}$ is therefore empty for RGA. Its ordinary-list ordering is chosen later by the sequential witness; it is not encoded in raw-state replay.

Lean: `RGA.RGAM_all_comm, RGA.RGAM_rc_either`.

4.3 Replay laws and convergence

Write

$$\begin{aligned} \mathsf{distinct}(a,b) &\iff \mathsf{time}(a) \neq \mathsf{time}(b), \\ \mathsf{differentReplicas}(a,b) &\iff \mathsf{rep}(a) \neq \mathsf{rep}(b). \end{aligned}$$

Thus distinct is timestamp inequality, which implies event inequality for observed events because replay contexts have unique timestamps. The update-layer bundle has three conjuncts. Directional noncommutation is

$$\begin{aligned} \mathsf{RcNonCommDirectional}(D) \iff \forall a,b.\ &\mathsf{distinct}(a,b) \wedge \mathsf{differentReplicas}(a,b) \\ &\implies \left(\neg(a \leftrightarrow b) \iff \begin{array}{l} \ \ (\mathsf{order}(a,b) = \mathsf{FstThenSnd}) \\ \vee\ (\mathsf{order}(b,a) = \mathsf{FstThenSnd}) \end{array} \right). \end{aligned}$$

Thus every distinct cross-replica conflict is oriented in at least one direction, and only conflicts receive such an orientation. The chain restriction is

$$\begin{aligned} \mathsf{NoRcChain}(D) \iff \forall a,b,c.\ &\mathsf{distinct}(a,b) \wedge \mathsf{distinct}(b,c) \\ &\implies \neg\big(\mathsf{order}(a,b) = \mathsf{FstThenSnd} \wedge \mathsf{order}(b,c) = \mathsf{FstThenSnd}\big). \end{aligned}$$

It rules out two consecutive first-then-second policy edges. Finally, conditional commutation lift is

$$\begin{aligned} \mathsf{CondCommLift}(D) \iff \forall s,e,e',e'',\pi.\ &\mathsf{distinct}(e,e') \wedge \mathsf{distinct}(e,e'') \wedge \mathsf{distinct}(e',e'') \\ &\wedge \mathsf{order}(e,e') = \mathsf{FstThenSnd} \wedge \neg(e' \leftrightarrow e'') \\ &\implies \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s,e'),e),\pi),e'') \\ &\qquad = \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s,e),e'),\pi),e''). \end{aligned}$$

Therefore

$$\mathsf{ReplayLaws}(D) \iff \mathsf{RcNonCommDirectional}(D) \wedge \mathsf{NoRcChain}(D) \wedge \mathsf{CondCommLift}(D).$$

These conjuncts correspond respectively to Lean's `rc_non_comm_directional`, `no_rc_chain`, and `cond_comm_lift` fields.

Lean: `ReplayLaws`.

With these laws, transitive and irreflexive visibility, and finite support, $\mathsf{loOn}_{R,E}$ is acyclic on E, admits a respecting enumeration, and all respecting enumerations fold to the same state. Intuitively, acyclicity says that the replay constraints are jointly schedulable; existence produces at least one such schedule; and convergence says that choosing a different valid schedule cannot change the result. Thus a finite event set E determines one replayed state even when it admits several valid orders.

For events in E, write

$$a \ \mathsf{loOn}^{\neq}_{R,E} \ b \iff a \neq b \wedge a \in E \wedge b \in E \wedge a \ \mathsf{loOn}_{R,E} \ b.$$

Write $\mathsf{TransGen}(Q)$ for the nonempty transitive closure of a relation Q.

Lemma 4.1 (Existence and uniqueness of set-relative replay)**.** *Suppose*

$$\mathsf{ReplayLaws}(D), \qquad \mathsf{Transitive}(R.\mathsf{vis}), \qquad \mathsf{Irreflexive}(R.\mathsf{vis}),$$
$$E \subseteq R.\mathsf{events}, \qquad \mathsf{listPermOf}(\ell, E).$$

Then:

1. $\mathsf{loOn}^{\neq}_{R,E}$ *is acyclic:*

$$\forall a.\ \neg\mathsf{TransGen}(\mathsf{loOn}^{\neq}_{R,E})(a, a).$$

2. E *has a respecting enumeration:*

$$\exists \rho.\ \mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E}).$$

3. all respecting enumerations of E *have the same fold, from every starting state:*

$$\begin{aligned} \forall s, \rho_1, \rho_2.\quad & \mathsf{listPermOf}(\rho_1, E) \wedge \mathsf{listPermOf}(\rho_2, E) \\ & \wedge \mathsf{respects}(\rho_1, \mathsf{loOn}_{R,E}) \wedge \mathsf{respects}(\rho_2, \mathsf{loOn}_{R,E}) \\ & \Longrightarrow \mathsf{applySeq}(D, s, \rho_1) = \mathsf{applySeq}(D, s, \rho_2). \end{aligned}$$

Proof sketch. For acyclicity, first observe that a policy edge cannot have a successor in $\mathsf{loOn}_{R,E}$. If the successor is a visibility edge, its endpoint is an absorber forbidden by the policy edge; if it is another policy edge, $\mathsf{NoRcChain}$ gives a contradiction. It follows inductively that every nonempty transitive $\mathsf{loOn}^{\neq}_{R,E}$ path is either a visibility path or ends with a policy edge that cannot be extended. A cycle of the first kind would give $a \ \mathsf{vis} \ a$ by transitivity, contradicting irreflexivity. A cycle of the second kind would extend its terminal policy edge by the first edge of the cycle, also a contradiction.

For existence, use ℓ as a finite enumeration of E. A walk through this list must find a $\mathsf{loOn}_{R,E}$-maximal event; otherwise following outgoing edges would revisit an event and create a cycle. Remove that event, recursively enumerate the remaining set, and append the maximal event. Repeating this construction produces a permutation that respects $\mathsf{loOn}_{R,E}$.

For uniqueness, induct on the length of one respecting enumeration. Let e be its head and locate e in the other enumeration, say $\rho_2 = \sigma +\!\!+ [e] +\!\!+ \tau$. Minimality of e in the first enumeration and respectfulness of the second make e incomparable with every event crossed while moving it left through σ. A commuting event swaps with e directly. For a noncommuting event y, same-replica visibility totality would make y and e comparable, so they must come from different replicas.

$\mathsf{RcNonCommDirectional}$ then orients their conflict. Since that orientation nevertheless creates no $\mathsf{loOn}_{R,E}$ edge, the definition of loOn supplies a later absorber $e_3 \in E$. Respectfulness places the absorber after the crossed pair, and $\mathsf{CondCommLift}$ shows that swapping y and e does not change the state after e_3 is applied. Bubble e to the front, remove it from both enumerations, and apply the induction hypothesis. The strengthened induction retains absorbers of surviving events, which is why no backward-closure premise on E is required. □

Lean: `loOnNe_acyclic_of_replayLaws, exists_loOn_respecting_perm_of_replayLaws, convergence_on_of_replayLaws`.

5 Canonical states and canonical configurations

Definition 5.1 (Canonical state)**.**

$$\begin{aligned}\mathsf{Canonical}(R, E, s) \iff \exists \rho.\ &\mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E})\\ &\wedge \mathsf{applySeq}(D, \sigma_0, \rho) = s.\end{aligned}$$

By set-relative convergence, the witness denotes the unique fold of every respecting enumeration, under the replay laws.

Lean: `Foundation.IsCanonicalState`.

Definition 5.2 (Canonical configuration)**.** Let $R_C = \mathsf{replayContext}(C)$. Then $\mathsf{CanonicalConfig}(C)$ consists of:

$$\begin{array}{ll}\mathsf{canonical}: & S(v) = (s, E) \Rightarrow \mathsf{Canonical}(R_C, E, s),\\ \mathsf{visTrans}: & a \mathbin{\mathsf{vis}} b \wedge b \mathbin{\mathsf{vis}} c \Rightarrow a \mathbin{\mathsf{vis}} c,\\ \mathsf{visIrrefl}: & \neg(a \mathbin{\mathsf{vis}} a),\\ \mathsf{supported}: & S(v) = (s, E) \wedge a \in E \Rightarrow a \in C.\mathsf{events},\\ \mathsf{causal}: & S(v) = (s, E) \wedge a \mathbin{\mathsf{vis}} b \wedge b \in E \Rightarrow a \in E,\end{array}$$

Lean: `CanonicalConfig`.

The invariant ranges over every allocated version, including historical merge bases. Its internal replay projection is

$$\begin{aligned}\mathsf{HasReplayWitness}(C) \iff \forall v, s, E.\ S(v) = (s, E) \to \exists \pi.\ &\mathsf{listPermOf}(\pi, E)\\ &\wedge \mathsf{respects}(\pi, \mathsf{lo}_C)\\ &\wedge \mathsf{applySeq}(D, \sigma_0, \pi) = s.\end{aligned}$$

Canonicality implies this property because respecting the stronger set-relative order implies respecting the global order:

$$\mathsf{CanonicalConfig}(C) \Longrightarrow \mathsf{HasReplayWitness}(C).$$

This establishes internal replay adequacy. Section 7 relates the replay witness to an independent client specification.

Lean: `HasReplayWitness, hasReplayWitness_of_canonical`.

6 Merge preservation

6.1 The semantic contract

For replay context R, abbreviate

$$\begin{aligned}
\mathsf{Context}(R) &\equiv \mathsf{Transitive}(\mathsf{vis}) \wedge \mathsf{Irreflexive}(\mathsf{vis}),\\
\mathsf{Supported}_R(E) &\equiv E \subseteq R.\mathsf{events},\\
\mathsf{WeakClosed}_R(E) &\equiv \forall a, b.\ a \ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b) \wedge b \in E \rightarrow a \in E.
\end{aligned}$$

Weak closure retains exactly the visible predecessors whose effects cannot be swapped past the event being replayed. Requiring full causal closure would also retain commuting predecessors, although their relative presence cannot change the fold. This is why the merge theorem closes histories under noncommuting visibility rather than under all of visibility.

Definition 6.1 (Join at a context)**.** `JoinAt D R` is

$$\begin{aligned}
\forall E_1, E_2, s_0, s_1, s_2.\ \mathsf{Context}(R) \wedge \bigwedge_{i=1,2} (\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i))&\\
\wedge\ \mathsf{Canonical}(R, E_1 \cap E_2, s_0) \wedge \mathsf{Canonical}(R, E_1, s_1) \wedge \mathsf{Canonical}(R, E_2, s_2)&\\
\Longrightarrow \mathsf{Canonical}(R, E_1 \cup E_2, \mathsf{merge}(s_0, s_1, s_2)).&
\end{aligned}$$

`Join D` is $\forall R.$ `JoinAt D R`.

Lean: `JoinAt, Join`.

Join is the load-bearing merge theorem: canonical intersection and branch states map to the canonical union state. Its context-local statement is reused by both ordinary and virtual-base executions.

Definition 6.2 (Restricted merge scope)**.** For a predicate $G : \mathsf{ReplayContext}(D) \rightarrow \mathsf{Prop}$,

$$\mathsf{JoinOn}(D, G) \Longleftrightarrow \forall R.\ G(R) \rightarrow \mathsf{JoinAt}(D, R).$$

Thus all-context `Join` has no configuration precondition, whereas `JoinOn` records one explicitly without saying how executions establish it. All-context Join supplies Join under every predicate, and $\mathsf{JoinOn}(D, \lambda\_.\mathsf{True})$ is equivalent to $\mathsf{Join}(D)$.

Lean: `JoinOn, Join.toJoinOn, joinOn_true_iff`.

These are the complete semantic interface for merge preservation. The remaining definitions in this section are reusable ways to prove all-context `Join`; they do not introduce further notions of merge correctness.

6.2 Proof routes for merge preservation

The reusable proof has one primary canonical-state contract. A datatype may prove that contract directly or obtain it from stronger equations over arbitrary states. It may also bypass the reusable induction and prove `Join` or `JoinOn` directly.

6.2.1 Canonical-state Join laws

For any nonempty finite family of event sets, write

$$\mathsf{Admissible}_R(E_1,\ldots,E_n) \equiv \mathsf{Context}(R) \wedge \bigwedge_{i=1}^{n}(\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i)),$$
$$\mathsf{Maximal}_{R,U}(e) \equiv e \in U \wedge \forall x \in U.\ x \neq e \Rightarrow \neg\mathsf{IoOn}_{R,U}(e,x).$$

Define the one-step noncommuting-visibility relation by $\mathsf{visNC}_R(a,b) \equiv a \ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b)$, and write visNC_R^+ for its positive transitive closure. The principal downset of e is

$$\downarrow_R e \equiv \{x \mid x = e \vee \mathsf{visNC}_R^+(x,e)\}.$$

We omit the subscript R below when the replay context is fixed. In particular, if $e \in E$ and $\mathsf{WeakClosed}_R(E)$, then $\downarrow_R e \subseteq E$.

The common core is

$$\begin{aligned}\mathsf{JoinCoreLaws}(D) = \langle &\mathsf{replay} : \mathsf{ReplayLaws}(D),\\ &\mathsf{mergeComm} : \forall l,a,b.\ \mathsf{merge}(l,a,b) = \mathsf{merge}(l,b,a)\rangle.\end{aligned}$$

Thus `JoinCoreLaws` contains replay and branch commutativity.

Lean: `JoinCoreLaws`.

The canonical delta component, `FeasibleDeltaLaws`, contains a unit law and the two delta equations restricted to the exact canonical tuples consumed by the Join induction. Below, R ranges over replay contexts, E, E_1, E_2, U over event sets, e over events, and $s, l, m, x, y, x_0, x_1, x_2$ over states.

$$\begin{aligned}\mathsf{init} :\quad \forall R,E,s.\quad &\mathsf{Supported}_R(E) \wedge \mathsf{Canonical}(R,E,s)\\ &\Longrightarrow \mathsf{merge}(\sigma_0,\sigma_0,s) = s.\end{aligned}$$

Thus the unit equation is required only for a state produced by a canonical replay of a supported event set.

For an event present only on the first branch,

$$\begin{aligned}&\mathsf{localRedistribute} :\\ &\forall R,E_1,E_2,l,m,x,y,e.\\ &\mathsf{Admissible}_R(E_1,E_2) \wedge e \in E_1 \setminus E_2 \wedge \mathsf{Maximal}_{R,E_1\cup E_2}(e)\\ &\wedge \mathsf{Canonical}(R,E_1\cap E_2,l) \wedge \mathsf{Canonical}(R,\downarrow e \setminus \{e\},m)\\ &\wedge \mathsf{Canonical}(R,E_1\setminus\{e\},x) \wedge \mathsf{Canonical}(R,E_2,y)\\ &\Longrightarrow \mathsf{merge}(l,\mathsf{merge}(m,x,\mathsf{do}(m,e)),y)\\ &\qquad = \mathsf{merge}(m,\mathsf{merge}(l,x,y),\mathsf{do}(m,e)).\end{aligned}$$

Intuitively, the equation extracts the maximal local event through the outer merge. The states l, m, x, and y represent respectively the branch intersection, the noncommuting dependency past of e, the first branch without e, and the second branch.

Figure 2 presents the two sides of `localRedistribute` as alternative factorizations of the same union history.

(a) Merge the local event into branch 1, then merge the branches

(b) Merge the histories without e, then merge in the local event

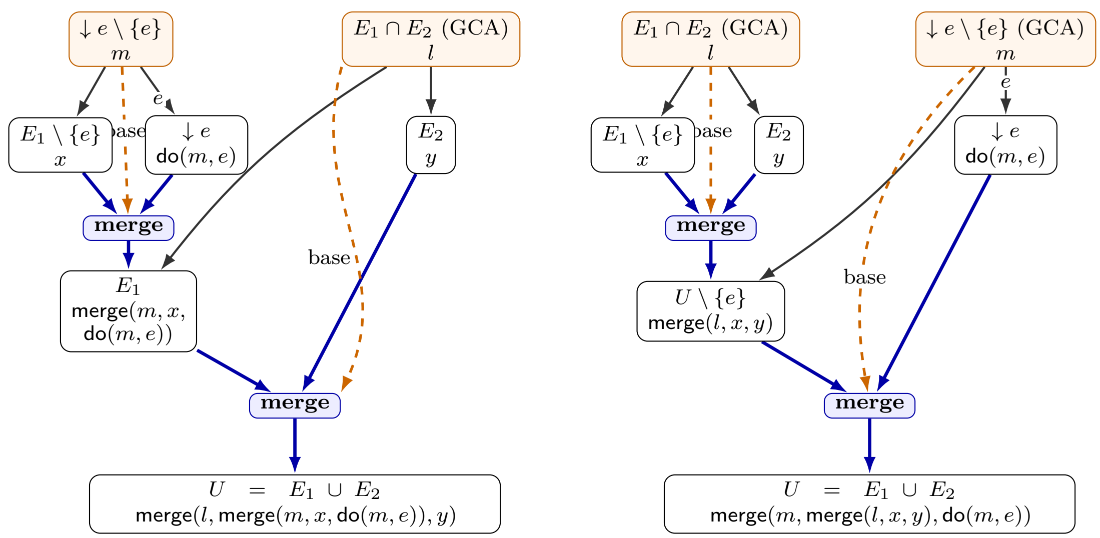

Figure 2: The local redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state to a gate, and orange boxes identify those GCA states. All boxes are proof-level canonical states; the law does not require all of them to be allocated versions in one execution. Panel (a) constructs the E_1 state before the outer merge. Panel (b) first constructs the $U \setminus \{e\}$ state and then merges in e. Both panels flow from top to bottom. `localRedistribute` identifies the two terminal states.

For an event shared by both branches,

$$
\begin{aligned}
&\mathsf{redistribute}:\\
&\forall R, E_1, E_2, m, x_0, x_1, x_2, e.\\
&\mathsf{Admissible}_R(E_1, E_2) \wedge e \in E_1 \cap E_2 \wedge \mathsf{Maximal}_{R, E_1 \cup E_2}(e)\\
&\wedge \mathsf{Canonical}(R, (E_1 \cap E_2) \setminus \{e\}, x_0) \wedge \mathsf{Canonical}(R, \downarrow e \setminus \{e\}, m)\\
&\wedge \mathsf{Canonical}(R, E_1 \setminus \{e\}, x_1) \wedge \mathsf{Canonical}(R, E_2 \setminus \{e\}, x_2)\\
&\Longrightarrow \mathsf{merge}(\mathsf{merge}(m, x_0, \mathsf{do}(m, e)), \mathsf{merge}(m, x_1, \mathsf{do}(m, e)), \mathsf{merge}(m, x_2, \mathsf{do}(m, e)))\\
&\qquad\qquad = \mathsf{merge}(m, \mathsf{merge}(x_0, x_1, x_2), \mathsf{do}(m, e)).
\end{aligned}
$$

This equation extracts one shared maximal event from all three merge components. The feasibility restrictions exclude representation states that no event history constructs.

Lean: `FeasibleDeltaLaws`.

Figure 3 presents the two sides of `redistribute` as alternative factorizations of the same union history.

The last component peels the selected maximal event from the target union.

$$
\begin{aligned}
\mathsf{CausalDeltaLaw}(D) \iff \forall R, U, A, B, e.\quad &\mathsf{Admissible}_R(U) \wedge \mathsf{Maximal}_{R,U}(e)\\
&\wedge \mathsf{Canonical}(R, U \setminus \{e\}, A) \wedge \mathsf{Canonical}(R, \downarrow e \setminus \{e\}, B)\\
&\Longrightarrow \mathsf{merge}(B, A, \mathsf{do}(B, e)) = \mathsf{do}(A, e).
\end{aligned}
$$

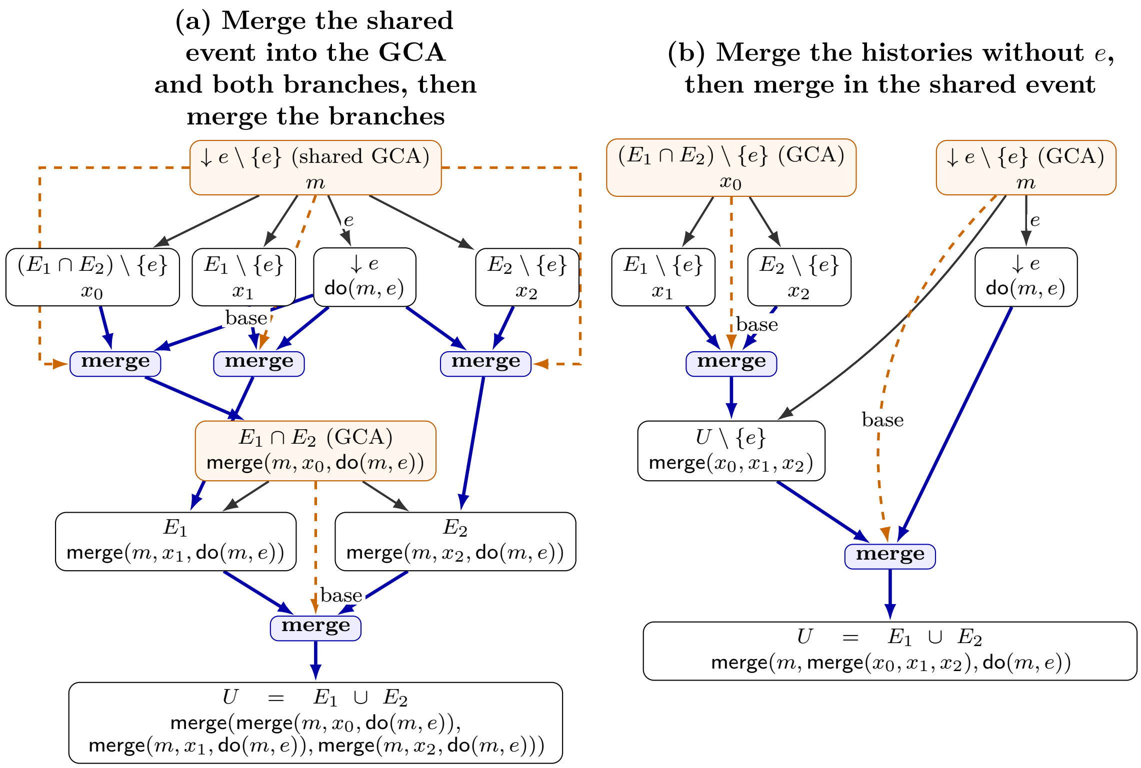

Figure 3: The shared redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state, and orange boxes identify GCA states. Panel (a) uses the same state m as the base of three inner merges, producing the branch intersection and the two branch states before the outer merge. Panel (b) first constructs $U \setminus \{e\}$ at base x_0, then merges it with $\downarrow e$ at base m. The fan-out edges in panel (a) reuse the same m and $\mathsf{do}(m, e)$ states. Both panels flow from top to bottom, and `redistribute` identifies their terminal states.

Here A represents the remaining history and B represents the event's noncommuting dependency past.

Lean: `CausalDeltaLaw`.

The single instance-facing bundle is

$$\begin{aligned}\mathsf{CanonicalJoinLaws}(D) = \langle &\mathsf{core} : \mathsf{JoinCoreLaws}(D),\\ &\mathsf{delta} : \mathsf{FeasibleDeltaLaws}(D),\\ &\mathsf{causalDelta} : \mathsf{CausalDeltaLaw}(D)\rangle.\end{aligned}$$

Its primary theorem is

$$\mathsf{CanonicalJoinLaws}(D) \Longrightarrow \mathsf{Join}(D).$$

The proof inducts on the union history. The empty case uses the canonical unit law. A maximal event local to one branch uses `localRedistribute`; a maximal event shared by both branches uses `redistribute`. Both cases use `CausalDeltaLaw` to peel the selected event.

Lean: `CanonicalJoinLaws`.

Lean: `CanonicalJoinLaws.join, JoinProof.ofFeasibleStateLaws`.

6.2.2 Universal equations as a shortcut

Many datatypes satisfy stronger equations for arbitrary representation states. The universal merge bundle `MergeLaws` contains only the fields used by the universal-to-canonical bridge:

$$\begin{aligned}\mathsf{MergeLaws}(D) = \langle &\mathsf{replay} : \mathsf{ReplayLaws}(D),\\ &\mathsf{mergeComm} : \forall l, a, b.\ \mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a),\\ &\mathsf{mergeInit} : \forall s.\ \mathsf{merge}(\sigma_0, \sigma_0, s) = s\rangle.\end{aligned}$$

Lean: `MergeLaws`.

The bundle `DeltaLaws` contains the two universal delta equations. Here c is an arbitrary state; the canonical adapter substitutes $\mathsf{do}(m, e)$ for it.

$$\begin{aligned}\mathsf{merge}(\mathsf{merge}(m, x_0, c), \mathsf{merge}(m, x_1, c), \mathsf{merge}(m, x_2, c)) &= \mathsf{merge}(m, \mathsf{merge}(x_0, x_1, x_2), c),\\ \mathsf{merge}(l, \mathsf{merge}(m, x, c), y) &= \mathsf{merge}(m, \mathsf{merge}(l, x, y), c).\end{aligned}$$

Lean: `DeltaLaws`.

The auxiliary `CommutingPeelLaw` states that an update can be pulled through a merge when it commutes with every event replayed in the other two components:

$$\begin{aligned}&\mathsf{CommutingPeelLaw}(D) \iff\\ &\quad \forall a, e, \pi_0, \pi_2. \quad (\forall x \in \pi_0.\ e \leftrightarrow x) \wedge (\forall x \in \pi_2.\ e \leftrightarrow x)\\ &\quad \implies \mathsf{merge}\big(\mathsf{applySeq}(D, \sigma_0, \pi_0), \mathsf{do}(a, e), \mathsf{applySeq}(D, \sigma_0, \pi_2)\big)\\ &\qquad = \mathsf{do}\Big(\mathsf{merge}\big(\mathsf{applySeq}(D, \sigma_0, \pi_0), a, \mathsf{applySeq}(D, \sigma_0, \pi_2)\big), e\Big).\end{aligned}$$

This auxiliary adapter sits outside the canonical Join contract. Together with `MergeLaws` and universal update commutativity, it supplies `CausalDeltaLaw` for the all-commuting instance class.

$$\begin{aligned}\mathsf{MergeLaws}(D) + \mathsf{CommutingPeelLaw}(D) + (\forall e, e'.\ e \leftrightarrow e')\\ \implies \mathsf{CausalDeltaLaw}(D).\end{aligned}$$

Lean: `CommutingPeelLaw`.

Lean: `causalDeltaLaw_of_all_comm`.

The universal equations construct the canonical contract:

$$\begin{aligned}\mathsf{MergeLaws}(D) &\implies \mathsf{JoinCoreLaws}(D),\\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) &\implies \mathsf{FeasibleDeltaLaws}(D),\\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) \wedge \mathsf{CausalDeltaLaw}(D) &\implies \mathsf{CanonicalJoinLaws}(D).\end{aligned}$$

Lean: `MergeLaws.toJoinCore`.

Lean: `feasibleDeltaLaws_of_delta, CanonicalJoinLaws.ofArbitrary`.

The bridge uses `MergeLaws` for the common core and canonical unit law, substitutes $\mathsf{do}(m, e)$ for the arbitrary delta-state argument in each `DeltaLaws` equation, and discards the canonical-tuple premises.

The adapter `JoinProof.ofArbitraryStateLaws` first constructs the canonical law bundle and invokes its Join theorem. Both proof routes therefore conclude the same `Join` property.

Lean: `JoinProof.ofArbitraryStateLaws`.

RGA instance. RGA takes the universal shortcut. Its pointwise Boolean merge proves `MergeLaws` and `DeltaLaws`; raw updates commute, so `CommutingPeelLaw` supplies `CausalDeltaLaw`. The resulting chain is

$$\mathsf{MergeLaws} + \mathsf{DeltaLaws} + \mathsf{CommutingPeelLaw} + \mathsf{allComm} \Longrightarrow \mathsf{CanonicalJoinLaws} \Longrightarrow \mathsf{Join}(\mathsf{RGAM}).$$

The issuer predicate `RGA.applicable` is used later to construct a legal ordinary-list history; the RGA merge theorem itself holds for every replay context.

Lean: `RGA.RGAM_mergeLaws, RGA.RGAM_deltaLaws, RGA.join`.

6.3 Ordinary adequacy

The initial configuration is canonical. Fork copies a canonical version; apply extends one with a fresh maximal event; query stutters; and merge is the Join implication after the GCA theorem identifies the base set with $E_1 \cap E_2$. Write $\mathsf{Reachable}_{\mathsf{Step}}(C_0, C)$ for the reflexive-transitive closure of the raw transition relation. Hence

$$\mathsf{Join}(D) \to \mathsf{Reachable}_{\mathsf{Step}}(\mathsf{initConfig}(D), C) \to \mathsf{HasReplayWitness}(C).$$

The proof establishes `CanonicalConfig C` first and projects replay only at the end.

Lean: `replayWitness_of_join`.

7 Issuance and client-facing correctness

7.1 Issuance-certified execution

An issuance policy contains

$$\mathsf{CanIssue} : \mathsf{Event}(O) \to \Sigma \to \mathsf{Prop}.$$

It constrains creation at the origin state and does not alter raw execution.

Lean: `Issuance`.

For a generation guard $G : \mathsf{Event}(O) \to \Sigma \to \mathsf{Prop}$, the provenance predicate is

$$\begin{aligned} \mathsf{MintHonest}(D, G, C) \iff \forall e \in C.\mathsf{events}.\ \exists \pi.\ & \mathsf{listPermOf}(\pi, \{e' \in C.\mathsf{events} \mid e' \ \mathsf{vis}\ e\}) \\ & \wedge \mathsf{respects}(\pi, \mathsf{vis}) \\ & \wedge G(e, \mathsf{applySeq}(D, \sigma_0, \pi)). \end{aligned}$$

Lean: `MintHonest`.

`IssuedStep D I` wraps a raw step. A non-apply label needs only its raw derivation. An apply $\mathsf{apply}(t, r, o)$ additionally requires

$$H(r) = v \wedge S(v) = (s, E) \wedge I.\mathsf{CanIssue}((t, r, o), s).$$

`MintCertifiedReach D I C` is the inductive closure from the initial configuration in which every step is issued and mint honesty is retained before and after it.

Lean: `IssuedStep, MintCertifiedReach`.

Certified reachability forgets to raw reachability:

$$\mathsf{MintCertifiedReach}(D, I, C) \Longrightarrow \mathsf{Reachable}_{\mathsf{Step}}(\mathsf{initConfig}(D), C).$$

Thus an all-context `Join` proof is independent of issuance: erase the issuance evidence and apply the ordinary adequacy theorem.

Lean: `MintCertifiedReach.toReachable`.

The second connection is explicit. For $G : \mathsf{ReplayContext}(D) \to \mathsf{Prop}$, define

$$\mathsf{IssuanceEstablishes}(D, I, G) \iff \forall C.\ \mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C) \to G(R_C).$$

This predicate says only that issuance consistency establishes the configuration condition used by a restricted merge proof.

Lean: `IssuanceEstablishes`.

The two axes compose as follows:

$$\begin{aligned} &\mathsf{JoinOn}(D, G) \land \mathsf{IssuanceEstablishes}(D, I, G) \\ &\qquad \to \mathsf{MintCertifiedReach}(D, I, C) \to \mathsf{HasReplayWitness}(C). \end{aligned}$$

The merge theorem supplies $G(R_C) \to \mathsf{JoinAt}(D, R_C)$; mint honesty supplies $G(R_C)$. Neither definition contains the other.

Lean: `replayWitness_of_mintCertified`.

7.2 Public interaction order

An interaction verdict is independent or conflicting. A concurrent conflict may require first-then-second, second-then-first, or no fixed order. Formally,

$$\begin{aligned} \mathsf{ConcurrentOrder} &= \{\mathsf{FstThenSnd}, \mathsf{SndThenFst}, \mathsf{Unconstrained}\}, \\ \mathsf{Interaction} &= \mathsf{Independent} \mid \mathsf{Conflict}(\mathsf{ConcurrentOrder}). \end{aligned}$$

The predicates used below are

$$\begin{aligned} i.\mathsf{Conflicts} &\iff \exists o.\ i = \mathsf{Conflict}(o), \\ i.\mathsf{FstBeforeSnd} &\iff i = \mathsf{Conflict}(\mathsf{FstThenSnd}). \end{aligned}$$

`InteractionSpec D` supplies

$$\mathsf{interaction} : \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Interaction}$$

and proves that reversing arguments flips the verdict.

Lean: `ConcurrentOrder, Interaction, InteractionSpec`.

Its set-relative public order is

$$\begin{aligned} e_1\ \mathsf{interactionLoOn}_{A,C,E}\ e_2 \iff &(e_1\ \mathsf{vis}\ e_2 \land A(e_1, e_2).\mathsf{Conflicts}) \\ &\lor (\neg(e_1\ \mathsf{vis}\ e_2) \land \neg(e_2\ \mathsf{vis}\ e_1) \land A(e_1, e_2).\mathsf{FstBeforeSnd} \\ &\qquad \land \neg\exists e_3 \in E.\ e_2\ \mathsf{vis}\ e_3 \land A(e_2, e_3).\mathsf{Conflicts}). \end{aligned}$$

It is structurally similar to loOn but independent of the proof-local policy and concrete universal commutation.

Lean: `interactionLoOn`.

7.3 Sequential specification and public theorem

A sequential specification contains

$$\begin{aligned}
\Sigma_S &: \mathsf{Type}, \\
\sigma_0^S &: \Sigma_S, \\
\mathsf{step}_S &: \Sigma_S \to \mathsf{Event}(O) \to \Sigma_S, \\
\mathsf{Legal}_S &: \mathsf{List}(\mathsf{Event}(O)) \to \mathsf{Prop}, \\
\mathsf{query}_S &: \Sigma_S \to Q \to R.
\end{aligned}$$

Its run is the left fold of step_S from σ_0^S. Legality speaks only about the abstract history; it cannot inspect implementation state.

Lean: `SequentialMachine, SequentialSpec, SequentialSpec.run`.

Definition 7.1 (Client-facing linearizability)**.** For $\mathsf{Rel} : \Sigma \to \Sigma_S \to \mathsf{Prop}$, `IsSpecLinearizable D A Spec Rel C` is

$$\begin{aligned}
\forall v, s, E.\ S(v) = (s, E) \to \exists \pi.\ &\mathsf{listPermOf}(\pi, E) \\
&\wedge \mathsf{respects}(\pi, \mathsf{interactionLoOn}_{A,R_C,E}) \\
&\wedge \mathit{Spec}.\mathsf{Legal}(\pi) \\
&\wedge \mathsf{Rel}(s, \mathit{Spec}.\mathsf{run}(\pi)) \\
&\wedge \forall q.\ D.\mathsf{query}(s, q) = \mathit{Spec}.\mathsf{query}(\mathit{Spec}.\mathsf{run}(\pi), q).
\end{aligned}$$

Lean: `IsSpecLinearizable`.

This requires a spec-legal exact history, relates implementation and abstract states, and equates every query. It is strictly the client theorem, whereas `HasReplayWitness` is an internal bridge.

8 Canonical virtual merge bases

An execution DAG need not have a GCA for every pair. The widened semantics supplies a synthetic base state computed from the antichain of maximal common ancestors. No synthetic base version is allocated.

8.1 Resolver and widened merge

A resolver V contains

$$V.\mathsf{state} : \mathsf{Configuration}(D) \to \mathsf{Version} \to \mathsf{Version} \to \Sigma.$$

`StepV D V` includes every ordinary step through `base` and one additional merge constructor. For head states s_1, s_2, its new-version equation is

$$S'(v_m) = (\mathsf{merge}(V.\mathsf{state}(C, v_1, v_2), s_1, s_2), E_1 \cup E_2),$$

with the same head, parent, freshness, and rank updates as ordinary merge. It does not require a GCA.

Lean: `VirtualMergeBaseResolver, StepV, StepV.mergeVirtual`.

`IssuedStepV D V I` embeds an ordinary issued step or accepts a genuinely virtual `StepV` derivation that is not an ordinary `Step`. `MintCertifiedReachV D V I C` is its inductive closure from the initial configuration, retaining mint honesty before and after every step.

Lean: `IssuedStepV, MintCertifiedReachV`.

8.2 Recursive maximal-ancestor fold

The recursive fold needs a generalization of the pairwise definition. For a finite support X, let

$$\begin{aligned}\mathsf{CAWithAny}_P(X, w) &= \bigcup_{u \in X} \mathsf{CommonAncestors}(P, u, w),\\ \mathsf{IsMaximalWithAny}_P(X, w, m) &\iff m \in \mathsf{CAWithAny}_P(X, w)\\ &\quad \wedge \forall x \in \mathsf{CAWithAny}_P(X, w).\ m \text{ Reaches } x \to x = m.\end{aligned}$$

In other words, a candidate must be a common ancestor of w and at least one member of X; it need not be an ancestor of every member of X. Lean names these definitions `CommonAncestorsWithAny` and `IsMaximalCommonAncestorWithAny`. Write

$$\mathsf{BasesWithAny}(P, X, w) = \{m \mid \mathsf{IsMaximalWithAny}_P(X, w, m)\}$$

for the finite enumeration implemented by `maximalCommonAncestorsWithAny`. The singleton bridge is

$$\mathsf{IsMaximalWithAny}_P(\{a\}, b, m) \iff \mathsf{IsMaximalCommonAncestor}(P, a, b, m).$$

Lean: `CommonAncestorsWithAny`, `IsMaximalCommonAncestorWithAny`, `isMaximalCommonAncestorWithAny_singleton`.

Lean: `maximalCommonAncestorsWithAny`, `mem_maximalCommonAncestorsWithAny_iff`.

Sort $\mathsf{BasesWithAny}(P, X, w)$ by increasing version rank. Let $\mathsf{stateD}(v)$ return the stored state, defaulting to σ_0 only to make the function total. The scratch fold is

$$\begin{aligned}\mathsf{vfold}(A, a, []) &= a,\\ \mathsf{vfold}(A, a, m :: ms) &= \mathsf{vfold}\big(A \cup \{m\}, \mathsf{merge}(\mathsf{vbase}(A, m), a, \mathsf{stateD}(m)), ms\big).\end{aligned}$$

The nested base is itself the fold of this sorted maximal antichain:

$$\mathsf{vbase}(X, w) = \begin{cases}\sigma_0, & \mathsf{sort}(\mathsf{BasesWithAny}(P, X, w)) = [],\\ \mathsf{vfold}(\{m_1\}, \mathsf{stateD}(m_1), ms), & \mathsf{sort}(\mathsf{BasesWithAny}(P, X, w)) = m_1 :: ms.\end{cases}$$

Termination uses the cardinality of the joint ancestor support, lexicographically paired with pending-list length. The head-pair resolver is

$$\mathsf{virtualMergeBaseState}(C, v_1, v_2) = \mathsf{vbase}(\{v_1\}, v_2).$$

The resolver whose state field is this function is $\mathsf{canonicalVirtualMergeBase}(D)$.

Lean: `vfoldAux`, `virtualBaseAux`, `virtualMergeBaseState`, `canonicalVirtualMergeBase`.

8.3 Virtual canonicity and adequacy

For finite $X \subseteq A_C$,

$$\mathsf{unionEvents}(C, X) = \bigcup_{v \in X} \mathsf{events}_C(v).$$

Under `StoreInv`, for allocated w, the maximal-base event union is exactly the outer intersection:

$$\mathsf{unionEvents}(C, \mathsf{BasesWithAny}(P, X, w)) = \mathsf{unionEvents}(C, X) \cap \mathsf{events}_C(w).$$

Lean: `unionEvents, maximalCommonAncestorsWithAny_unionEvents`.

The fold theorem states

$$\begin{aligned}&\mathsf{StoreInv}(S,P)\wedge \mathsf{CanonicalConfig}(C)\wedge \mathsf{JoinAt}(D,R_C)\\&\wedge S(v_1)=(s_1,E_1)\wedge S(v_2)=(s_2,E_2)\\&\Longrightarrow \mathsf{Canonical}(R_C,E_1\cap E_2,\mathsf{virtualMergeBaseState}(C,v_1,v_2)).\end{aligned}$$

Lean: `virtualMergeBaseState_canonical`.

Consequently,

$$\begin{aligned}&\mathsf{Join}(D)\to \mathsf{Reachable}_{\mathsf{StepV}}(\mathsf{initConfig}(D),C)\to \mathsf{CanonicalConfig}(C),\\&\mathsf{Join}(D)\to \mathsf{Reachable}_{\mathsf{StepV}}(\mathsf{initConfig}(D),C)\to \mathsf{HasReplayWitness}(C).\end{aligned}$$

Lean: `canonicalConfig_reachableV, replayWitnessV_of_join`.

As in the ordinary semantics, certified widened reachability erases to raw widened reachability. Hence all-context Join needs no issuance premise. The restricted form is

$$\begin{aligned}&\mathsf{JoinOn}(D,G)\wedge \mathsf{IssuanceEstablishes}(D,I,G)\\&\quad\to \mathsf{MintCertifiedReachV}(D,\mathsf{canonicalVirtualMergeBase}(D),I,C)\to \mathsf{HasReplayWitness}(C).\end{aligned}$$

Lean: `MintCertifiedReachV.toReachable`.

Lean: `replayWitness_of_mintCertifiedV`.

9 Verification certificates and instances

9.1 How an instance closes the theorem stack

A replay-adequacy certificate has field

$$\mathsf{soundV}: \quad \mathsf{MintCertifiedReachV}(D,\mathsf{canonicalVirtualMergeBase}(D),I,C)\to \mathsf{HasReplayWitness}(C).$$

Ordinary soundness follows because ordinary execution embeds in widened execution.

Lean: `ReplayAdequacyCertificate, ReplayAdequacyCertificate.sound`.

The artifact provides one constructor for each merge-proof route:

$$\begin{aligned}\mathsf{Join}(D)&\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D,I),\\ \mathsf{JoinOn}(D,G)\wedge \mathsf{IssuanceEstablishes}(D,I,G)&\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D,I).\end{aligned}$$

The first constructor erases certified widened reachability before applying all-context Join; the second uses issuance consistency to discharge the explicit context condition.

Lean: `ReplayAdequacyCertificate.ofJoin`.

Lean: `ReplayAdequacyCertificate.ofJoinOn`.

The first constructor applies to an all-context merge theorem and accepts any issuance policy. The second applies to a restricted theorem: the local issuance predicate must establish the global replay-context condition G. This gives the instance-level dependency

$$\mathsf{CanIssue}\Longrightarrow \mathsf{MintHonest}\Longrightarrow G(R_C)\Longrightarrow \mathsf{JoinAt}(D,R_C)\Longrightarrow \mathsf{HasReplayWitness}(C).$$

The sequential certificate then chooses a legal ordering of that same event set and relates its abstract state and observations to the stored state.

Let $\mathsf{CertifiedExecution}(D, I, C)$ mean either an ordinary $\mathsf{MintCertifiedReach}(D, I, C)$ derivation or a widened $\mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)$ derivation. A sequential-correctness certificate has type

$$\forall C.\ \mathsf{CertifiedExecution}(D, I, C) \to \mathsf{HasReplayWitness}(C) \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C).$$

Lean: `CertifiedExecution, SequentialCorrectnessCertificate`.

The complete package is

$$\begin{aligned}\mathsf{VerifiedMRDT}(D) = (I &: \mathsf{Issuance}(D),\\ A &: \mathsf{InteractionSpec}(D),\\ K &: \mathsf{ReplayAdequacyCertificate}(D, I),\\ \mathit{Spec} &: \mathsf{SequentialSpec}(D),\\ \mathsf{Rel} &: \Sigma \to \Sigma_S \to \mathsf{Prop},\\ K_S &: \mathsf{SequentialCorrectnessCertificate}(D, I, A, \mathit{Spec}, \mathsf{Rel})).\end{aligned}$$

`PackagedMRDT` additionally contains a name and the signature D.

Lean: `VerifiedMRDT, PackagedMRDT`.

Its public conclusions are

$$\begin{gathered}\mathsf{MintCertifiedReach}(D, I, C) \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C),\\ \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)\\ \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C).\end{gathered}$$

Lean: `VerifiedMRDT.correct, VerifiedMRDT.correctV`.

9.2 Released-instance summary

The typed production registry contains 18 `PackagedMRDT` values. The table groups packages that instantiate the same parametric proof. "Join" means $\mathsf{Join}(D)$ from Section 6.1; "JoinOn G" means $\mathsf{JoinOn}(D, G)$. These are theorem scopes, while the final column names the public sequential specification.

RDT / released package	Merge theorem	Issuance policy	Sequential specification
grow-only-set, add-store, finite-add-store	Join	True	Grow-only set/store, total legality
counter, increment-only-counter, pn-counter	Join	True	Integer accumulator, total legality
flat-grow-only-set, flat-grow-only-map	Join	True	Pointwise Boolean store, total legality
bounded-counter	Join	`bcApplicable`	Rights-respecting integer counter
rga	Join	`RGA.applicable`	Legal ordinary-list machine

RDT / released package	Merge theorem	Issuance policy	Sequential specification
or-set	Join	`ORSet.canIssue`	Ordinary add-wins finite set
tree-move	Join	`applicable`	Chronological mutable tree
aegis-sheet	Join	`applicable`	Causal-origin spreadsheet machine
embed-rga, peritext-embed-rga	JoinOn `EHonestCore`	`eApplicable`	Legal embedded-list machine
sided-embed-rga	JoinOn `SHonestCore`	`sApplicable`	Legal sided-list machine
sided-peritext-core, sided-peritext-rich-core	JoinOn projected `SHonestCore`	`coreGuard`	Legal rich-editor machine

Lean: `Production.registry, Production.names`.

9.3 Formal instance profiles

Each profile below records the information needed to interpret the package: the representation, issuance predicate, merge scope, public sequential state and legality predicate, representation relation, and proof route. Helper lemmas remain in the cited instance modules.

Grow-only stores and flat counters. For AddStore and FinsetStore,

$$\Sigma = \mathcal{P}(\alpha) \text{ or } \mathsf{Finset}(\alpha), \qquad \mathsf{do}(s, e) = s \cup \{e.\mathsf{op}\}, \qquad \mathsf{merge}(l, a, b) = a \cup b.$$

For the counter family parameterized by $\delta : O \to \mathbb{Z}$,

$$\Sigma = \mathbb{Z}, \qquad \mathsf{do}(s, e) = s + \delta(e.\mathsf{op}), \qquad \mathsf{merge}(l, a, b) = a + b - l.$$

In both families, $I.\mathsf{CanIssue} = \mathsf{True}$, $\mathsf{Legal}(\pi) = \mathsf{True}$, the abstract machine repeats the displayed update, and Rel is equality. The pointwise-Boolean grow-only set and map use the same profile coordinatewise. All prove universal `MergeLaws`, `DeltaLaws`, and `CommutingPeelLaw`, derive all-context Join, and package total sequential correctness.

Lean: `AddStore.D, AddStore.generation, AddStore.spec, AddStore.verified`.

Lean: `FlatCounters.D, FlatCounters.generation, FlatCounters.spec, FlatCounters.verified`.

Bounded counter. The state records per-replica increment and decrement counts, $\Sigma = (\mathbb{N} \to \mathbb{Z})^2$, and

$$\mathsf{bcApplicable}(e, s) \iff \begin{cases} \mathsf{True}, & e.\mathsf{op} = \mathsf{inc}, \\ s_2(e.\mathsf{rep}) + 1 \le s_1(e.\mathsf{rep}), & e.\mathsf{op} = \mathsf{dec}, \end{cases}$$

$$\mathsf{ClientLegal}(\pi) \iff (\exists \rho.\ \pi = \mathsf{canonical}(\rho)) \wedge \forall r.\ \#\mathsf{dec}_r(\pi) \le \#\mathsf{inc}_r(\pi).$$

The sequential state is $\mathbb{N} \to \mathbb{Z}$; its value at r is increments minus decrements at r. The relation requires the representation invariant $0 \le s_2(r) \le s_1(r)$ and equality with those differences. Join is all-context by the universal-law route; issuance establishes sequential legality and the separate safety certificate.

Lean: `BoundedCounter.bcApplicable`, `BoundedCounter.ClientLegal`, `BoundedCounter.clientSpec`, `BoundedCounter.verified`.

Observed-remove set. For element type α,

$$\Sigma = \mathsf{Finset}(\mathsf{Timestamp} \times \alpha) \times \mathsf{Finset}(\mathsf{Timestamp}),$$

where the first component stores tagged additions and the second stores removed tags. Its issuer contract is

$$\mathsf{canIssue}(e, s) \iff \begin{cases} e.\mathsf{time} \notin \mathsf{tags}(s_1), & e = \mathsf{add}(x), \\ T = \mathsf{liveTags}(s, x), & e = \mathsf{remove}(x, T). \end{cases}$$

The public sequential state is $\mathsf{Finset}(\alpha)$; add inserts the element, remove erases it, $\mathsf{Legal}(\pi) = \mathsf{True}$, and

$$\mathsf{stateRel}(s, A) \iff \forall x.\ \mathsf{contains}(s, x) = [x \in A].$$

The MRDT effectors commute and prove all-context Join through the universal laws. Issuance is load-bearing for the tagged-storage refinement, even though the ordinary-set legality predicate is total: it proves that every remove payload contains exactly the tags observed at its causal origin. The public interaction order places a concurrent same-element remove before add, giving add-wins behavior.

Lean: `ORSet.canIssue`, `ORSet.interaction`, `ORSet.spec`, `ORSet.stateRel`, `ORSet.verified`.

Tombstone RGA. The representation and merge were introduced in Section 2. Its issuer rule is

$$\begin{aligned}
&\mathsf{applicable}((t, r, \mathsf{addAfter}(a, i)), s) \\
&\quad \iff i = t \wedge (a = 0 \vee \exists t', p.\ t' < t \wedge s_1(t', p, a) = \mathsf{true}) \\
&\qquad \wedge (\forall a', i'.\ s_1(t, a', i') = \mathsf{false}) \wedge (\forall t', a'.\ s_1(t', a', i) = \mathsf{false}) \wedge s_2(i) = \mathsf{false}, \\
&\mathsf{applicable}((t, r, \mathsf{remove}(i)), s) \iff (\exists t', a.\ s_1(t', a, i) = \mathsf{true}) \wedge s_2(i) = \mathsf{false}.
\end{aligned}$$

The public state is $\mathsf{List}(\mathbb{N})$. Its legality predicate requires unique timestamps, requires every insertion identifier to equal its timestamp and name root or an earlier insertion as anchor, and requires every removal to name an earlier insertion. The relation is $\mathsf{read}(s) = xs$. RGA proves all-context Join through the universal laws; issuance establishes well-formedness of the ordinary-list witness.

Lean: `RGA.applicable`, `RGA.generation`, `RGA.join`.

Lean: `RGA.listLegal`, `RGA.listSpec`, `RGA.listRel`, `RGA.verified`.

TreeMove. The state is a finite set of timestamped moves. Rendering sorts the set and applies each move to an ordinary tree. Issuance requires

$$\begin{aligned}
\mathsf{applicable}(e, s) \iff\ & \mathsf{freshKey}_s(e) \wedge \mathsf{visibleNode}_s(e.\mathsf{parent}) \\
& \wedge (\neg\mathsf{knownNode}_s(e.\mathsf{child}) \vee \mathsf{visibleNode}_s(e.\mathsf{child})) \\
& \wedge e.\mathsf{child} \neq \mathsf{root} \wedge e.\mathsf{child} \neq \mathsf{trash}.
\end{aligned}$$

The sequential state is a mutable tree, the step applies one move, and $\mathsf{Legal}(\pi) = \mathsf{Nodup}(\pi) \wedge \mathsf{Pairwise}(\mathsf{eventLE}, \pi)$. The relation equates the rendered event set with the sequential tree. Join is all-context by the universal-law route; the package also contains a separate tree-safety certificate.

Lean: `TreeMove.applicable`, `TreeMove.SequentialLegal`, `TreeMove.sequentialSpec`, `TreeMove.verified`.

AegisSheet. The MRDT stores a finite set of spreadsheet-intent events and merges by union. At issuance,

$$\mathsf{applicable}(e, E) \iff \mathsf{applicableB}(e, E) = \mathsf{true} \wedge \forall t \in \mathsf{eventTimes}(E).\ t < e.\mathsf{time}.$$

The Boolean part checks freshness, exact causal-origin metadata, action metadata, direct-action preconditions, and undo provenance. The public sequential state is the independent incremental spreadsheet state, and

$$\begin{aligned}\mathsf{CausalOriginLegal}(\pi) \iff &\ \mathsf{Chronological}(\pi) \wedge \\ &\forall \pi_0, e, \pi_1.\ \pi = \pi_0 +\!\!+ e :: \pi_1 \to \exists E_0 \subseteq \mathsf{toFinset}(\pi_0).\ \mathsf{applicable}(e, E_0).\end{aligned}$$

The relation states that every chronological enumeration materializes the same incremental state, that this state equals the event-set materialization, and that both answer the same sheet query. Join is all-context by universal laws; issuance establishes causal-origin legality and the safety invariants.

Lean: `AegisSheet.applicable`, `AegisSheet.generation`, `AegisSheet.join`.

Lean: `AegisSheet.Sequential.CausalOriginLegal`, `AegisSheet.Sequential.clientSpec`, `AegisSheet.verified`.

EmbedRGA and Peritext. An EmbedRGA insertion carries a payload, a coordinate prefix p, and an anchor a; deletion names an identifier. Its issuer rule is

$$\mathsf{eApplicable}(e, s) \iff \begin{cases} a < e.\mathsf{time} \wedge ((a = 0 \wedge p = []) \vee \exists x.\ (a, x, p) \in s), & e = \mathsf{ins}(x, p, a), \\ i \in \mathsf{ids}(s), & e = \mathsf{del}(i). \end{cases}$$

$\mathsf{EHonestCore}(R)$ requires each deletion target to have a visible insertion and requires all insertion coordinates to arise from one positive, timestamp-summing chain assignment. The direct theorem is $\mathsf{JoinOn}(E, \mathsf{EHonestCore})$, and mint honesty for `eApplicable` establishes that condition. The public sequential state is $\mathsf{List}(\mathbb{N} \times \alpha)$; legality requires increasing fresh insertion identifiers, the carried coordinate of an earlier anchor, and an earlier allocation for every deletion. The relation maps live records to identifier– payload pairs. Peritext instantiates the payload with characters and mark boundaries and inherits this certificate; its additional rendering theorem maps the certified buffer to marked text.

Lean: `EmbedRGA.EHonestCore`, `EmbedRGA.eApplicable`, `EmbedRGA.e_join_at`.

Lean: `ProductionRGA.embedLegal`, `ProductionRGA.embedClientSpec`, `ProductionRGA.embed`.

Lean: `Peritext.verified`.

SidedEmbedRGA and SidedPeritext. SidedEmbedRGA adds a left/right choice to insertion. Its `sApplicable`, `SHonestCore`, list legality, and relation are the sided analogues of the preceding profile; the side is selected at issuance and does not strengthen the anchor guard. Its merge theorem is $\mathsf{JoinOn}(S, \mathsf{SHonestCore})$.

SidedPeritext composes the sided sequence with grow-only deletion and mark stores. For an event e and state (s_T, s_D, s_M), its cross-component guard is

$$\mathsf{coreGuard}(e, s_T, s_D, s_M) \iff \begin{cases} \mathsf{sApplicable}(e, s_T), & e = \mathsf{textIns}, \\ \mathsf{False}, & e = \mathsf{textDel}, \\ i \in \mathsf{ids}(s_T) \wedge i \notin s_D, & e = \mathsf{deleteStoreAdd}(i), \\ \mathsf{validMark}(e, s_T, s_D, s_M), & e = \mathsf{markStoreAdd}. \end{cases}$$

The merge theorem composes projected $\mathsf{SHonestCore}$ with the two all-context grow-only-store theorems. The sequential state contains the sided list and both stores; legality is exactly the projected sided-list legality, while the issuance guard validates cross-component delete and mark references. The rich package changes only the query boundary to rendered marks.

Lean: `SidedEmbedRGA.SHonestCore`, `SidedEmbedRGA.sApplicable`, `SidedEmbedRGA.s_join_at`.

Lean: `SidedPeritext.coreGuard`, `SidedPeritext.core_join_at`, `SidedPeritext.clientSpec`, `SidedPeritext.verified`, `SidedPeritext.richVerified`.

9.4 Proof routes and evidence status

Released family	Merge-proof route
Grow-only stores, flat counters, bounded counter, OR-Set, TreeMove, AegisSheet, RGA	Universal equations adapted to `CanonicalJoinLaws`, then all-context `Join`
EmbedRGA and SidedEmbedRGA	Direct datatype-specific `JoinAt` under `EHonestCore` or `SHonestCore`
Peritext	Inherited payload-parametric EmbedRGA certificate
SidedPeritext	Componentwise `joinAt_prod`: restricted sided sequence plus two all-context grow-only stores

The negative and partial-evidence ledger is separate from the production registry. The multi-value register has replay adequacy but a mechanized counterexample to the ordinary single-register specification. The queue has conditional replay adequacy but a duplicate-dequeue FIFO counterexample. The FugueMax signature proves internal ordering-policy facts but has no public `VerifiedMRDT`. These artifacts remain useful formal evidence, but they are not released datatype packages.

Lean: `Instances.MVR.concurrentState_no_sequential_register`, `Instances.Queue.ConditioningSPOT.duplicate_dequeue_not_fifo`, `Instances.SidedEmbedRGA.fuguemax_replay_witness`.

10 Distributed commit-history collection

Replicas fetch commits asynchronously, so each replica decides locally when history can be removed. It must retain enough authored evidence and enough graph information for future reachability and merge-base queries.

10.1 Local holdings and evidence

A local holding is

$$\mathsf{Local} = (\mathsf{head} : \mathsf{Version}, \mathsf{commits} : \mathcal{P}(\mathsf{Version}), \mathsf{headPresent} : \mathsf{head} \in \mathsf{commits}).$$

An envelope contains a commit set. Advertising exports the local set; receiving unions it into the destination without changing the head. A world is $\mathsf{World} = \mathsf{Replica} \to \mathsf{Local}$.

Lean: `GC.Local, GC.Envelope, GC.advertise, GC.receive, GC.World`.

Authorship has type $\mathsf{Author} = \mathsf{Version} \to \mathsf{Option}(\mathsf{Replica})$. Evidence for roster member r is

$$\begin{aligned}\mathsf{DerivedEvidence}(P, A, L, r) = \{v \mid v \in L.\mathsf{commits} \wedge A(v) = r \\ \wedge \mathsf{Reaches}(P, v, L.\mathsf{head})\}.\end{aligned}$$

Evidence is complete for roster R at replica r_s when

$$\begin{aligned}\mathsf{EvidenceComplete}(P, A, R, r_s, L) \iff \forall r \in R.\ r = r_s \vee \\ \exists v.\ v \in \mathsf{DerivedEvidence}(P, A, L, r).\end{aligned}$$

Lean: `GC.Author, GC.DerivedEvidence, GC.EvidenceComplete`.

10.2 Compression certificate

For any reachability relation R_V on versions, define

$$\begin{aligned}\mathsf{IsGCARel}(R_V, v_1, v_2, v_T) \iff R_V(v_T, v_1) \wedge R_V(v_T, v_2) \\ \wedge \forall w.\ R_V(w, v_1) \wedge R_V(w, v_2) \to R_V(w, v_T).\end{aligned}$$

The canonical compressed edge summarizes any old path between retained endpoints:

$$\mathsf{compressedEdge}_{P,K}(a, b) \iff a \in K \wedge b \in K \wedge \mathsf{Reaches}(P, a, b).$$

Define

$$\mathsf{CompressedReaches}_{P,K} = \left(\mathsf{compressedEdge}_{P,K}\right)^*.$$

This relation preserves reachability exactly on retained nodes. Closure of K under maximal common ancestors suffices to preserve GCA answers; retaining intermediate paths or version 0 is unnecessary.

Lean: `GC.compressedEdge, GC.CompressedReaches, GC.compressedReaches_iff`.

Lean: `GC.compressed_isGCA_iff_of_maximalCommonAncestorClosed, GC.root_absent_when_dropped`.

A collection certificate chooses $K \subseteq L.\mathsf{commits}$ and proves:

1. evidence is complete, the local head is retained, and suitable evidence for every remote roster member is retained;

2. for $a, b \in K$, $\mathsf{CompressedReaches}_{P,K}(a, b)$ is equivalent to $\mathsf{Reaches}(P, a, b)$; and
3. for retained v_1, v_2, v_T, $\mathsf{IsGCARel}(\mathsf{CompressedReaches}_{P,K}, v_1, v_2, v_T)$ is equivalent to $\mathsf{IsGCA}(P, v_1, v_2, v_T)$.

Collection replaces the local commit set by K and preserves its head.

Lean: `GC.Certificate, GC.collect`.

Define the pairwise closure condition by

$$\mathsf{MaximalClosed}_P(K) \iff \forall a, b \in K.\ \forall m.\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) \to m \in K.$$

The canonical constructor then has the schematic type

$$\begin{aligned}\mathsf{ofMaximalClosed} : \mathsf{EvidenceComplete}(P, A, R, r_s, L) &\to L.\mathsf{head} \in K \\ &\to (\forall r \in R.\ r = r_s \vee \exists v \in \mathsf{DerivedEvidence}(P, A, L, r).\ v \in K) \\ &\to K \subseteq L.\mathsf{commits} \to (\forall v, p.\ p \in P(v) \to p < v) \\ &\to \mathsf{MaximalClosed}_P(K) \to \mathsf{Certificate}.\end{aligned}$$

The exact Lean constructor is named in the source anchor.

Lean: `GC.Certificate.ofMaximalCommonAncestorClosed`.

10.3 Asynchronous refinement

The collecting protocol has fetch, head synchronization, and local collection. Its comparison protocol has fetch and head synchronization but no collection. Define

$$\begin{aligned}\mathsf{StoreSim}(F, C) &\iff F.\mathsf{head} = C.\mathsf{head} \\ &\qquad \wedge C.\mathsf{commits} \subseteq F.\mathsf{commits} \wedge C.\mathsf{head} \in C.\mathsf{commits}, \\ \mathsf{WorldSim}(F, C) &\iff \forall r.\ \mathsf{StoreSim}(F(r), C(r)).\end{aligned}$$

One collecting step is matched by the corresponding network step or stuttering. Write Steps_{GC} and $\mathsf{NoGCSteps}$ for the reflexive-transitive closures of the collecting and comparison step relations, respectively. Then

$$\begin{aligned}&\mathsf{WorldSim}(F_0, C_0) \wedge \mathsf{Steps}_{GC}(C_0, C_1) \\ &\qquad \Longrightarrow \exists F_1.\ \mathsf{NoGCSteps}(F_0, F_1) \wedge \mathsf{WorldSim}(F_1, C_1).\end{aligned}$$

Lean: `GC.StoreSim, GC.WorldSim, GC.execution_refines_noGC`.

11 Runtime and datatype-state collection

11.1 Commit-store runtime refinement

A datatype-rich runtime pairs semantics with local stores:

$$\mathsf{Runtime}(D) = (\mathsf{core} : \mathsf{Configuration}(D), \mathsf{stores} : \mathsf{World}).$$

It is well formed when each retained commit is allocated in the core and each semantic replica head agrees with its store head. A visible runtime transition contains an actual `Step` derivation and matching store evolution. Fetch and collection are silent.

Lean: `GC.Runtime, GC.Runtime.WellFormed, GC.RuntimeStep`.

Erasing optional labels removes silent steps:

$$\mathsf{eraseLabels} = \mathsf{List.filterMap}(\mathsf{id}).$$

Let RuntimeSteps be the list-labelled reflexive-transitive closure of runtime transitions, and let CoreSteps be the corresponding closure of raw `Step` transitions. Finite runtime traces refine raw semantic traces:

$$\mathsf{RuntimeSteps}(D, A, R, S_0, ls, S_1) \to \mathsf{CoreSteps}(D, S_0.\mathsf{core}, \mathsf{eraseLabels}(ls), S_1.\mathsf{core}).$$

The widened theorem uses the virtual-merge-base step relations and parameterizes availability by the exact versions read by the resolver.

Lean: `GC.eraseLabels, GC.runtime_refines_core, GC.runtime_refines_coreV`.

11.2 Datatype-state GC

A local collector is first specified independently of a runtime transition system. For an issuance policy I, a certificate supplies compact states C, collection evidence W, a relation c Rep s to the full datatype state, an evidence predicate, and compact interpretations of initialization, update, ternary merge, and query. Its central obligations are

$$\begin{aligned} c \ \mathsf{Rep}\ s \wedge \mathsf{EvidenceValid}(w, c, s) &\Longrightarrow \mathsf{collect}(w, c)\ \mathsf{Rep}\ s, \\ c \ \mathsf{Rep}\ s \wedge I.\mathsf{CanIssue}(e, s) &\Longrightarrow \mathsf{update}_C(c, e)\ \mathsf{Rep}\ \mathsf{do}(s, e), \\ c_l \ \mathsf{Rep}\ l \wedge c_a \ \mathsf{Rep}\ a \wedge c_b \ \mathsf{Rep}\ b \wedge \mathsf{Compatible}(a, b) &\Longrightarrow \mathsf{merge}_C(c_l, c_a, c_b)\ \mathsf{Rep}\ \mathsf{merge}(l, a, b), \\ c \ \mathsf{Rep}\ s &\Longrightarrow \mathsf{query}_C(c, q) = \mathsf{query}(s, q). \end{aligned}$$

Together with the corresponding initialization law, this is `StateGCCertificate`. The generic `StateGCCertificate.exactState` chooses $C = \Sigma$, equality as Rep, and identity collection. It is a preservation baseline, not a reclamation result.

The more general `StateGCProtocol` embeds collection into physical execution. It supplies

$$\begin{aligned} \mathsf{Physical} &: \mathsf{Type}, \\ \mathsf{semantic} &: \mathsf{Physical} \to \mathsf{Configuration}(D), \\ \mathsf{Valid} &: \mathsf{Physical} \to \mathsf{Prop}, \\ \mathsf{PhysicalStep} &: \mathsf{Physical} \to \mathsf{Option}(\mathsf{Label}(D)) \to \mathsf{Physical} \to \mathsf{Prop}. \end{aligned}$$

It proves validity preservation, silent stuttering $\mathsf{semantic}(P') = \mathsf{semantic}(P)$, and visible refinement to `StepV D V`. Write $\mathsf{SemanticSteps}_V$ for the list-labelled reflexive-transitive closure of `StepV D V`. Every finite valid physical trace therefore erases to such a widened semantic trace.

Lean: `StateGCCertificate, StateGCCertificate.exactState, StateGCProtocol, StateGCProtocol.refines`.

11.3 Per-datatype collection profiles

The typed `Production.StateGC.registry` follows the same eighteen-entry order as the released-instance registry. Its four constructors distinguish an exact-state baseline, a representation-changing local certificate, an operational protocol, and a staged collector. Thus registry coverage means that the status of every released package is explicit; it does not mean that every package already reclaims state.

Grow-only stores and flat grow-only stores. Grow-only set, AddStore, FinsetStore, flat grow-only set, and flat grow-only map currently use the semantic set or pointwise map as the compact state:

$$C = \Sigma, \qquad c \;\mathsf{Rep}\; s \iff c = s, \qquad \mathsf{collect}((), c) = c.$$

Their registry entries therefore carry the exact-state certificate. Every stored member is part of the monotone semantic value under the current signature; a stronger result would have to prove a representation quotient or state formally that no internal metadata is reclaimable.

Counters. The integer counter, increment-only counter, PN-counter, and bounded counter also carry exact-state certificates. Their states are already aggregates, not replay logs: one integer for the flat counters, or per-replica increment and decrement components for the bounded counter. Removing a component would need an explicit replica-retirement or coordinate-normalization protocol; none is part of the released semantics.

Tombstone RGA. RGA is staged because its event representation retains both insertion records and deletion tombstones. The intended collector needs stable-frontier evidence authorizing removal of a dead insertion while retaining enough anchor and order information for concurrent descendants and stale branches. Formally, it must instantiate C, W, and Rep above and prove update, ternary-merge, list-query, and continuation preservation. Until those obligations are discharged, its registry entry contains only the exact-state baseline.

EmbedRGA and Peritext. EmbedRGA, its Peritext specialization, and their current live-sequence representations use exact-state certificates. Deletion is represented by removal from the live sequence rather than a separate graveyard, but surviving coordinates and anchors may still constrain later insertions. A smaller physical encoding therefore needs a continuation theorem for coordinate and anchor summaries, not merely equality of the current rendered list.

SidedEmbedRGA and SidedPeritext core. The sided sequence and the ordinary SidedPeritext core likewise use exact state. In particular, the core query exposes its component stores, so a collector that deletes entries cannot preserve that query by equality. A representation-changing theorem must either retain those stores, change the public query boundary, or relate them through an explicit abstraction. The released rich-query package takes the latter route.

Rich SidedPeritext. The rich package has an operational collector. A compact state contains the retained sided text, a Fugue gap summary, deletion and mark stores, and a stable Lamport cut. Its representation evidence records a keep predicate, knowledge used to reconstruct the full semantic text, gap-map correctness, an old-identifier bound, and equality of every rich-text query. The physical system can silently collect text, trim deletion metadata, drop a cancelling mark pair, or reframe identifiers; ordinary and virtual-base operations remain visible. `StateGCProtocol.refines` then covers arbitrary finite interleavings of those steps.

Lean: `SidedPeritext.StateGC.CompactState`, `SidedPeritext.StateGC.collectText`, `SidedPeritext.StateGC.trimDeleted`, `SidedPeritext.StateGC.dropMarkPair`.

Lean: `SidedPeritext.StateGC.Protocol.Represents`, `SidedPeritext.StateGC.Protocol.protocol`.

TreeMove. TreeMove has an operational collector with

$$C = \{\mathsf{base} : \mathsf{Tree},\ \mathsf{pending} : \mathsf{List}(\mathsf{Event})\}, \qquad \mathsf{materialize}(c) = \mathsf{replayFrom}(c.\mathsf{base}, c.\mathsf{pending}).$$

A `StableCut` splits canonical replay into a stable prefix and pending suffix. Collection folds the prefix into the base tree; after the pending suffix is empty, a second collector prunes the hidden trash subtree. The representation relation equates the compact visible tree with the query of the full event set, and the packaged protocol proves silent collection plus visible ordinary and widened execution.

Lean: `TreeMove.GC.StableCut`, `TreeMove.GC.Compact`, `TreeMove.GC.collectPrefix`, `TreeMove.GC.collectTrash`, `TreeMove.GC.protocol`.

AegisSheet. The current AegisSheet MRDT stores an event set and merges it by union. Its collector retains replicated purge markers but removes exactly the covered cell payloads:

$$\mathsf{semanticCollect}(E) = \{e \in E \mid e \text{ is not a cell update covered by a retained marker}\}.$$

The compact state is another finite event set, and

$$c\ \mathsf{Rep}\ E \iff c = \mathsf{semanticCollect}(E) \wedge \mathsf{StateValid}(E) \wedge \mathsf{TimestampUnique}(E).$$

The certificate normalizes after collection, update, and union merge and proves equality of the whole sheet view. This is genuine payload reclamation, but it is still a collector for the event-set encoding; it is not yet the proposed materialized three-way AegisSheet representation.

Lean: `AegisSheet.GC.semanticCollect`, `AegisSheet.GC.Represents`, `AegisSheet.GC.certificate`.

Observed-remove set. OR-Set is staged. Its full state retains tagged additions and removed-tag tombstones. A collector must use stable-frontier or equivalent evidence to remove a dead tag while proving that a stale branch cannot resurrect it and that future remove payloads and add-wins queries remain correct. The current registry entry deliberately supplies only the exact-state baseline.

Only AegisSheet, TreeMove, and rich SidedPeritext currently package representation-changing proofs. The roadmap treats every remaining exact-state profile as an open classification obligation: supply a real collector when metadata is reclaimable, or prove that the released representation contains no semantically redundant internal state.

Lean: `StateGCCoverage`, `PackagedStateGC`, `Production.StateGC.registry`, `Production.StateGC.names`.

11.4 Composition

The combined physical state pairs a datatype-state physical value with a commit world. A step is either a datatype-state step, with matching store evolution for a visible label, or a silent history fetch/collection step. Write $\mathsf{Combined.Valid}$ for componentwise validity and $\mathsf{CombinedSteps}$ for the list-labelled closure of these combined steps. `combinedProtocol` is itself a `StateGCProtocol`; hence

$$\begin{aligned}&\mathsf{Combined.Valid}(P) \wedge \mathsf{CombinedSteps}(P, ls, P') \\ &\qquad \Longrightarrow \mathsf{SemanticSteps}_V(\mathsf{semantic}(P), \mathsf{eraseLabels}(ls), \mathsf{semantic}(P')).\end{aligned}$$

If the datatype protocol also proves that every visible physical step is raw, the ordinary refinement follows.

Lean: `GC.combinedProtocol`, `GC.CombinedSteps.refinesV`, `GC.CombinedSteps.refinesRaw`.

A Exact correspondence index

The companion Lean ledger imports these modules and checks the named declarations, so a rename or type-level disappearance fails the document gate.

Concept	Lean declaration	Source
MRDT interface	`MRDTSig`	`Signature.lean`
Events and internal replay support	`Op`, `UpdateSig`	`UpdateSignature.lean`
Replay lists	`applySeq`, `listPermOf`, `respects`	`Replay.lean`
Operational semantics	`Configuration`, `Label`, `Step`, `StepV`	`Execution.lean`
Store/GCA events	`StoreInv`, `gca_events_of_storeInv`	`StoreInvariant.lean`
Set-relative replay	`loOn`, `IsCanonicalState`	`SetRelativeReplay.lean`
Replay laws	`ReplayLaws`	`ReplayLaws.lean`
Merge semantics	`JoinAt`, `Join`, `JoinOn`	`MergeLaws.lean`
Canonical-state Join laws	`CanonicalJoinLaws`, `CanonicalJoinLaws.join`; universal adapter `CanonicalJoinLaws.ofArbitrary`	`MergeLaws.lean`; `Adequacy.lean`
Canonical reachability	`CanonicalConfig`, `replayWitness_of_join`	`Adequacy.lean`
Issuance/spec interfaces	`Issuance`, `InteractionSpec`, `SequentialSpec`	`Certificates.lean`
Certified adequacy	`replayWitness_of_mintCertified`	`CertifiedAdequacy.lean`
Client theorem	`IsSpecLinearizable`, `VerifiedMRDT`	`Correctness.lean`
Virtual construction	`virtualMergeBaseState`	`VirtualMergeBase.lean`
Virtual adequacy	`virtualMergeBaseState_canonical`, `replayWitnessV_of_join`	`VirtualAdequacy.lean`
History GC	`GC.Certificate`, `GC.execution_refines_noGC`	`Distributed.lean`
Compressed DAG	`GC.CompressedReaches`	`CompressedDAG.lean`
Runtime refinement	`GC.runtime_refines_coreV`	`Refinement.lean`
Local state-GC certificate	`StateGCCertificate`	`StateGC.lean`
Datatype-state GC protocol	`StateGCProtocol`	`StateGC.lean`
Per-datatype GC coverage	`Production.StateGC.registry`	`StateGCCoverage.lean`
GC composition	`GC.CombinedSteps.refinesV`	`StateComposition.lean`

B Historical binary boundary and negative evidence

The merge-free projection D_U from Section 2 is implemented by `UpdateSig`. The projection `MRDTSig.toUpdateSig` forgets queries and ternary merge, retaining only the data needed for event application, finite replay, and commutation. It is the internal algebraic view consumed by replay proofs.

Some historical proof modules still study a binary operation. They request it through a separate optional capability:

$$\mathsf{HistoricalBinaryMerge}(D_U) = (\mathsf{merge}_2 : \Sigma \to \Sigma \to \Sigma).$$

Lean exposes the operation as `UpdateSig.historicalMerge`, keeping it visibly distinct from the ternary `MRDTSig.merge`. For an MRDT D, the compatibility instance `MRDTSig.historicalBinaryMerge` supplies the initial-state slice

$$\mathsf{merge}_2(a, b) = D.\mathsf{merge}(\sigma_0, a, b).$$

Binary merge is therefore neither a field of `UpdateSig` nor part of the live execution or `VerifiedMRDT` interfaces. It appears only when a historical theorem explicitly requests it.

Lean: `Foundation.UpdateSig, Foundation.HistoricalBinaryMerge, Foundation.UpdateSig.historicalMerge`.

Lean: `MRDTSig.toUpdateSig, MRDTSig.historicalBinaryMerge`.

Historical binary VC bundles and countermodels are not premises of the public correctness theorem. Two negative results are retained as audit controls:

1. convergence over a merely backward-closed version set can fail when the absorber search ranges over a later whole configuration; and
2. associativity, commutativity, and idempotence of binary merge do not by themselves imply the corrected contextual ternary Join property.

They are machine-checked in negative-evidence modules and remain outside the production package.

Lean: `Foundation.convergence_over_backward_closed_subsets_false`.

Lean: `Foundation.binaryLaws_insufficient`.

C Notation and elaboration notes

- The paper writes S, H, and L as partial maps. Lean totalizes them with `Option`: $S(v) = (s, E)$ abbreviates `C.ver v = some (s,E)`, and $H(r) = v$ abbreviates `C.head r = some v`.
- $f(x) = \perp$ means $x \notin \mathsf{dom}(f)$; for a totalized Lean map, this is equality with `none`.
- Lean sets have type `Set alpha`. Finiteness is normally witnessed by a list satisfying `listPermOf`, rather than stored in the set type.
- $v < v'$ in allocation rules is the natural-number rank of version identifiers. Event Lamport time is the first component of an event and is constrained separately.
- The companion ledger uses the actual Lean namespaces and checks elaboration of every principal declaration cited here.